

Introdução à Computação

debugando erros na gerência de memória

Alexandre Rademaker

debugging I

memory.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void) {
5      int *x = malloc(3 * sizeof(int));
6      x[0] = 72;
7      x[1] = 73;
8      x[2] = 33;
9
10 }
```

Temos o que chamamos de 'memory leak'. Como detectar estes vazamentos?

debugging II

Leaks no MacOS

<https://youtu.be/bhhDRm926qA>

Valgrind e GDB no Linux

<https://youtu.be/8JEEYwdrexc>

Ferramentas dependem do sistema operacional, versão do compilador e dialeto da linguagem sendo usado. Muita documentação em <https://clang.llvm.org>.

debugging III

No MacOS

```
1 clang -g -I/usr/local/include memory.c -lm -o memory
2 leaks --list --atExit -- ./memory
3 export MallocStackLogging=1
4 leaks --list --atExit -- ./memory
5 unset MallocStackLogging
```

Linha 1 adiciona a chamada do compilador o parâmetro `-g`. A linha 3 declara uma variável de ambiente que nos permite encontrar a posição do vazamento (linha do código).

lixo na memória I

Vejam isso

garbage0.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(void) {
5     int scores[3];
6     for (int i = 0; i < 3; i++)
7         printf("%i ", scores[i]);
8     printf("\n");
9 }
```

Que valores serão impressos?

lixo na memória II

Vamos examinar este código

garbage1.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(void) {
5
6     int *x = NULL;
7     int *y = NULL;
8     x = malloc(sizeof(int));
9
10    *x = 42;
11    *y = 13;
12    y = x;
13    *y = 13;
14
15    printf("teste x->%p y->%p x=%d y=%d\n", x, y, *x, *y);
16    free(x);
17    printf("teste x->%p y->%p x=%d y=%d\n", x, y, *x, *y);
18 }
```

Quais erros temos aqui? ... uma dica

Problemas comuns

heap overflow se usarmos `malloc` para pedir muita memória

stack overflow se chamarmos muitas funções em cadeia sem retornar delas (tipicamente funções recursivas que se chamam)

Veja também **buffer overflow**.

Exemplos extras

Arquivos em `src/` relacionados ao tema desta semana, para estudo complementar (não foram apresentados em aula):

▶ `size.c`